

FreelanceHub

Freelance Marketplace Platform

DOCUMENT TYPE

Software Engineering Project Report

VERSION

1.0

TECH STACK

React · Node.js · PostgreSQL · Prisma

DATE

May 2026

TEAM MEMBERS

- Saksham Sontakke
- Harsh Verma
- Ajit Kumar Prasad
- Sankalp
- Kartik Mehra

Table of Contents

01	Executive Summary
02	Problem Statement & Solution
03	How the System Works
04	User Roles & Responsibilities
05	Key Features
06	Technology Stack & Architecture
07	Security & Data Privacy
08	Performance & Scalability
09	Project Metrics & Codebase Overview
10	Future Roadmap

01

Executive Summary

FreelanceHub is a comprehensive, full-stack freelance marketplace platform designed to bridge the gap between skilled freelancers and clients seeking quality project delivery. Built on a modern technology stack — React, Node.js, Express, PostgreSQL, and Prisma ORM — the platform provides an end-to-end digital workspace that streamlines the entire freelance engagement lifecycle.

The system provides an intuitive interface for clients to post projects, evaluate applications, and manage deliverables, while enabling freelancers to discover opportunities, submit competitive bids, and grow their professional portfolios. A secure JWT-based authentication mechanism and granular role-based access control ensure each user category — Client, Freelancer, and Administrator — interacts only with the resources and workflows appropriate to their role.

Platform Highlights

- Full-stack marketplace: React frontend + Node.js/Express REST API + PostgreSQL database
- Role-based access control: distinct workflows for Clients, Freelancers, and Admins
- Advanced discovery engine with search, multi-attribute filtering, sorting & pagination
- Secure application/bidding workflow with cover-letter submission
- Fully responsive UI with Light/Dark mode theming
- Production-ready REST API with modular, middleware-protected routes

02

Problem Statement & Solution

The Problem

The global freelance economy has grown exponentially, yet existing marketplace platforms continue to frustrate both clients and freelancers with legacy UX patterns and missing discovery features. The core pain points identified are:

- **Complex & cluttered interfaces:** Existing platforms overwhelm users with poorly structured dashboards and non-intuitive navigation flows.
- **Weak project discovery:** Lack of intelligent filtering and matching means freelancers waste time browsing irrelevant listings.
- **Limited search capabilities:** Rigid search bars with no multi-attribute filtering (skills, budget range, category, date) reduce match quality.
- **Poor freelancer discoverability:** Clients cannot easily filter the talent pool by skills, experience, or availability.
- **Fragmented workflows:** Application management, messaging, and project tracking are siloed across different parts of the platform.

The FreelanceHub Solution

FreelanceHub was architected from the ground up to address each of these pain points with deliberate engineering decisions:

Problem	FreelanceHub Solution
Cluttered UI	Clean component-based React UI with intuitive navigation and theme support
No intelligent discovery	Multi-attribute search, category/skill filters, budget sort, and pagination
Rigid search	Composable query parameters handled server-side via Prisma query builders
Poor freelancer visibility	Dedicated freelancer profiles with skill tags and filterable browse page
Fragmented workflows	Unified project lifecycle: post → bid → review → manage in one platform

03

How the System Works

FreelanceHub follows a standard three-tier web architecture. The React frontend communicates exclusively through Axios HTTP calls to the Express.js REST API. The API applies authentication middleware, validates input, and delegates all data operations to Prisma ORM, which interacts with a PostgreSQL relational database.

Architecture Flow

Layer	Component	Responsibility
Presentation	React (v19)	UI rendering, state management, routing, API integration
Transport	Axios + REST	HTTP communication, auth headers, error handling
API	Express.js	Route handling, JWT middleware, request validation
ORM	Prisma	Type-safe database queries, schema migrations, relations
Database	PostgreSQL	Persistent storage of users, projects, applications

Core Workflows

Client Workflow:

- Register/Login → receives JWT token
- Create a project with title, description, budget, category, and required skills
- Browse incoming applications with bid amounts and cover letters
- Accept or reject applications and manage project lifecycle

Freelancer Workflow:

- Register/Login → receives JWT token with Freelancer role
- Browse the project marketplace with search, filter, sort, and pagination
- Submit a bid application including bid amount and cover letter
- Track application status and manage profile/portfolio

04

User Roles & Responsibilities

FreelanceHub implements a strict role-based access control (RBAC) model. Each user is assigned one of three roles at signup, and every API endpoint validates the role before executing any operation.

Role	Core Responsibilities	Key Permissions
Client	Post and manage projects Review and manage applications Select winning freelancers	CRUD on own projects Read all freelancer profiles View/manage applications for own projects
Freelancer	Browse the project marketplace Submit bids with cover letters Manage own profile	Read all projects Create/view own applications Update own profile
Admin	Platform governance and oversight Content moderation User management	Read access across all entities Moderate flagged content Manage user accounts

Role enforcement is applied at the API middleware level. Every protected route first validates the JWT token and then checks the decoded role claim against an allowlist for that endpoint. Requests from unauthorized roles receive an HTTP 403 Forbidden response before any business logic executes.

05

Key Features

Category	Features
Authentication & Auth	JWT-based signup/login, bcrypt password hashing, role-based route guards, token expiry (7d)
Project Management	Full CRUD for projects, rich project metadata (budget, category, skills), client-only mutation
Advanced Discovery	Full-text search across title/description, filter by category & skills, sort by budget/date, cursor-based pagination
Application & Bidding	Freelancers submit bid amount + cover letter, clients view all applications per project, application status tracking
Freelancer Profiles	Skill tagging, profile browsing, filterable freelancer directory
Modern UI/UX	React v19, React Router DOM, responsive layout, Light/Dark mode, custom CSS design system
REST API	Modular Express router structure, middleware-protected endpoints, structured JSON responses, error handling

06

Technology Stack & Architecture

Layer	Technology	Version / Notes
Frontend	React	v19 — latest stable, concurrent features
Routing	React Router DOM	Client-side SPA routing
HTTP Client	Axios	Interceptors for JWT injection
Styling	Custom CSS	Light/Dark mode, responsive grid
Runtime	Node.js	v18+ recommended
API Framework	Express.js	Modular router, middleware chain
ORM	Prisma	Type-safe queries, schema-first migrations
Database	PostgreSQL	Relational DB, ACID transactions
Authentication	JWT + bcrypt	jsonwebtoken, 7d expiry, bcrypt salt rounds
Deployment (FE)	Vercel	vercel.app — serverless frontend hosting
Deployment (BE)	Render	render.com — Node.js service hosting

Database Schema (Key Entities)

The data model centres on three primary entities: **User**, **Project**, and **Application**. A User (Client) can own many Projects. A User (Freelancer) can submit many Applications. Each Application belongs to exactly one Project and one Freelancer, forming a many-to-many relationship between Users and Projects mediated by the Application join entity.

Entity	Key Fields	Relations
User	id, name, email, passwordHash, role (CLIENT/FREELANCER/ADMIN), skills, bio	Has many Projects (as client) Has many Applications (as freelancer)
Project	id, title, description, budget, category, skills[], status, clientId	Belongs to User (client) Has many Applications

Application	id, bidAmount, coverLetter, status, projectId, freelancerId	Belongs to Project Belongs to User (freelancer)
-------------	---	--

07

Security & Data Privacy

Security is enforced through multiple complementary layers spanning authentication, authorisation, data handling, and transport.

Authentication — JWT

- Stateless JSON Web Tokens signed with a server-held **JWT_SECRET**
- Token expiry set to **7 days** (configurable via `JWT_EXPIRES_IN`)
- Tokens are validated on every protected request via Express middleware
- Expired or tampered tokens result in immediate HTTP 401 Unauthorized

Password Security — bcrypt

- All passwords are hashed with **bcrypt** before storage — plaintext is never persisted
- bcrypt's adaptive cost factor makes brute-force attacks computationally expensive
- Password comparison uses constant-time `bcrypt.compare` to resist timing attacks

Role-Based Access Control

- Each API endpoint declares an **allowedRoles** list checked by middleware
- Role is encoded in the JWT payload and verified server-side on every request
- Clients cannot access freelancer-only routes and vice versa

Data Privacy

- Sensitive config (DB URL, JWT secret) stored exclusively in **.env** files — never committed to source control
- **.env.example** files document required variables without exposing real values
- User email addresses and personal data are stored only in the server-side database
- CORS policy restricts API access to whitelisted frontend origins

08

Performance & Scalability

FreelanceHub is designed with scalability considerations woven into the core architecture, enabling the platform to handle growing user bases and project volumes without fundamental refactoring.

Pagination

- All list endpoints support **page** and **limit** query parameters
- Server returns only the requested page slice — prevents unbounded queries
- Frontend renders paginated result sets with navigation controls

Efficient Database Queries

- Prisma generates optimised SQL with selective field projection (**select**)
- Indexes on frequently queried columns (projectId, freelancerId, category) improve lookup speed
- Relational joins are performed in-database rather than in application memory

Stateless API Design

- JWT-based auth means the API server holds no session state — horizontally scalable
- Multiple API server instances can run behind a load balancer without sticky sessions

Frontend Performance

- React v19 concurrent rendering minimises blocking UI updates
- Axios request cancellation prevents stale responses from unmounted components
- Component-level state management avoids unnecessary global re-renders

Deployment Architecture

- **Frontend:** deployed on Vercel — globally distributed CDN edge network
- **Backend:** deployed on Render — auto-scaling Node.js service
- Environment-specific API URLs configured via .env for seamless local/production switching

09

Project Metrics & Codebase Overview

API Endpoint Summary

Method	Endpoint	Description	Access
POST	/api/auth/signup	Register new user	Public
POST	/api/auth/login	Login, receive JWT	Public
GET	/api/projects	List projects (search/filter/sort/page)	Public
POST	/api/projects	Create a project	Client
GET	/api/projects/:id	Get project details	Public
PUT	/api/projects/:id	Update project	Client (owner)
DELETE	/api/projects/:id	Delete project	Client (owner)
POST	/api/applications	Submit bid application	Freelancer
GET	/api/applications/project/:id	Get applications for project	Client (owner)

Codebase Structure

The project is organised as a monorepo with clearly separated **frontend/** and **backend/** directories, each with its own package.json and environment configuration.

Path	Contents
backend/src/routes/	Modular Express routers per resource (auth, projects, applications)
backend/src/middleware/	JWT verification and role-check middleware
backend/src/controllers/	Business logic handlers called by routes
backend/prisma/schema.prisma	Database schema: User, Project, Application models
frontend/src/pages/	Top-level React page components (Home, Projects, Profile...)

frontend/src/components/	Reusable UI components (Cards, Forms, Filters, Pagination)
frontend/src/services/	Axios API service functions per resource

10

Future Roadmap

FreelanceHub has a strong foundational architecture that positions it well for iterative feature expansion. The following enhancements are planned across short-term, mid-term, and long-term horizons.

Short-Term (1–3 months)

- **Real-time messaging:** WebSocket-based chat between clients and freelancers
- **Notification system:** In-app and email notifications for application status changes
- **Enhanced freelancer profiles:** Portfolio uploads, work samples, and reviews
- **Application status management:** Clients can shortlist, accept, or reject applications with reasons

Mid-Term (3–6 months)

- **Payment integration:** Escrow-based payment gateway (Stripe/Razorpay) for secure transactions
- **Review & rating system:** Post-project ratings for both clients and freelancers
- **AI-powered matching:** Recommendation engine to surface best-fit projects/freelancers
- **Advanced analytics dashboard:** Earning trends, application success rates, project stats

Long-Term (6–12 months)

- **Mobile application:** React Native app for iOS and Android
- **Dispute resolution:** Structured mediation workflow for project disagreements
- **Subscription tiers:** Premium plans with enhanced visibility and reduced platform fees
- **API marketplace:** Expose FreelanceHub data via a public API for third-party integrations

FreelanceHub represents a modern, maintainable approach to freelance marketplace engineering — combining clean architecture, secure authentication, advanced discovery, and an extensible codebase ready to scale with growing demand.